Sri Sivasubramaniya Nadar College of Engineering, Chennai

(An Autonomous Institution affiliated to Anna University)

Degree & Branch	B.E Computer Science & Engineering	**Semester**	VI
Subject Code & Name	UCS2612 – Machine Learning Algorithms Laboratory		
Academic Year	2025–2026 (Even)	**Batch**	2023–2027
Name	Kamalesh A	**Section**	CSE-B
Registration No.	3122235001063	**Date**	28-02-2026

Experiment 5

Perceptron vs Multilayer Perceptron with Hyperparameter Tuning

1. Aim and Objective

Aim

To implement and compare the performance of a Single-Layer Perceptron Learning Algorithm (PLA) and a Multilayer Perceptron (MLP) with hyperparameter tuning for handwritten character recognition.

Objective

- To implement the Perceptron Learning Algorithm (PLA) using a step activation function.
- To extend PLA to multiclass classification using the One-vs-Rest strategy.
- To implement a Multilayer Perceptron (MLP) with hidden layers and nonlinear activation functions.
- To tune hyperparameters such as learning rate, batch size, optimizer, and activation function.
- To evaluate and compare both models using standard performance metrics.

2. Dataset Description

The English Handwritten Characters Dataset consists of grayscale images of handwritten characters used for classification tasks.

- **Total Samples:** 3,410
- **Number of Classes:** 62 (digits 0–9, uppercase letters A–Z, and lowercase letters a–z)
- **Image Type:** Grayscale
- **Image Size:** 28×28 pixels
- **Download Link:** english-handwritten-characters-dataset

Dataset Format

- `english.csv` containing image file names and corresponding class labels
- `Img/` directory containing the respective image files

3. Preprocessing

The following preprocessing steps were performed to prepare the dataset for model training and evaluation:

- Images were resized to 28×28 pixels.
- Images were converted to grayscale.
- Pixel values were normalized to the range $[0, 1]$.
- Images were flattened into 784-dimensional feature vectors.
- Class labels were encoded into numerical values.
- The dataset was split into training and testing sets in an 80:20 ratio.

Preprocessing ensures that the data is in a suitable format for both the Perceptron Learning Algorithm (PLA) and the Multilayer Perceptron (MLP) models.

4. PLA Implementation and Results

The Single-Layer Perceptron Learning Algorithm (PLA) was implemented from scratch using Python to classify handwritten characters. Since PLA is inherently a binary classifier, it was extended to handle multiclass classification using the One-vs-Rest (OvR) strategy.

Implementation Steps

- The preprocessed dataset consisting of flattened and normalized image vectors was used as input to the perceptron model.
- Model weights were initialized to zero, and a bias term was incorporated into the perceptron.
- For each input sample, a linear combination of inputs and weights was computed as:

$$z = \mathbf{w} \cdot \mathbf{x} + b \tag{1}$$

- A step activation function was applied to the linear output to generate binary predictions.
- Whenever a sample was misclassified, the perceptron weight update rule was applied as:

$$\mathbf{w}_{t+1} = \mathbf{w}_t + \eta(y - \hat{y})\mathbf{x} \tag{2}$$

- Since the dataset contains multiple classes, the PLA was extended using the One-vs-Rest (OvR) strategy by training one binary perceptron for each class.

- During training, the misclassification rate per epoch was recorded to analyze the convergence behavior of the model.
- After training, the PLA model was evaluated using the test dataset.

PLA Results

The performance of the PLA model was evaluated using the following metrics and visualizations:

Confusion Matrix

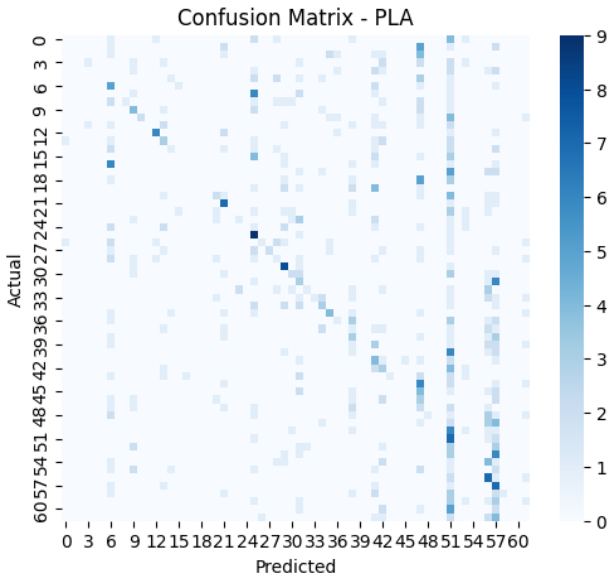

Figure 1: Confusion Matrix for PLA Model

ROC Curve (Micro and Macro Average) ROC curves were plotted using decision scores instead of probability estimates due to the use of a step activation function.

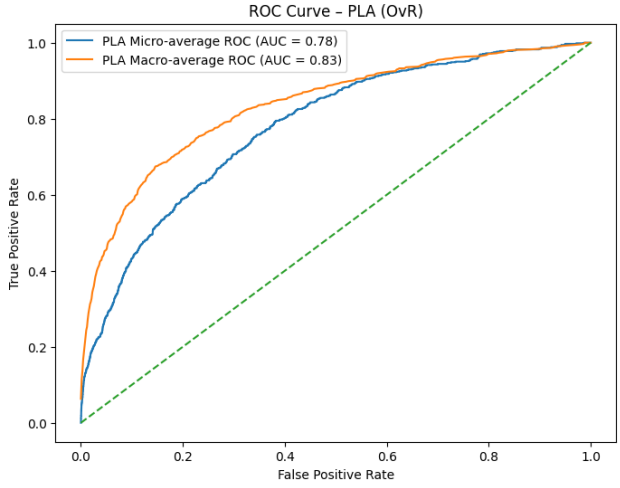

Figure 2: ROC Curve (Micro and Macro Average) for PLA

Training Loss vs Epochs Curve The misclassification rate per epoch was used as a loss metric to visualize the training behavior of the PLA model.

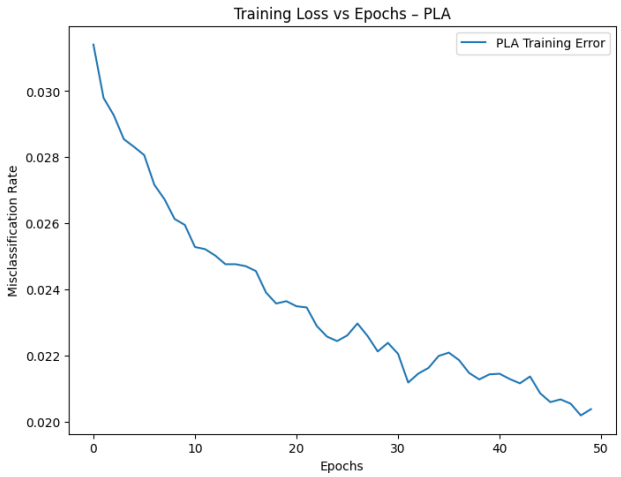

Figure 3: Training Loss vs Epochs for PLA

5. MLP Implementation and Results

MLP Implementation

The Multilayer Perceptron (MLP) was implemented to classify handwritten characters by learning nonlinear decision boundaries. Unlike the Single-Layer Perceptron, the MLP employs hidden layers and nonlinear activation functions, enabling it to achieve higher accuracy on complex image data.

Implementation Steps

- The preprocessed dataset consisting of normalized and flattened image vectors was used as input to the MLP model.
- An input layer with 784 neurons was defined, corresponding to the 28×28 pixel image size.
- Two hidden layers were added with 256 and 128 neurons, respectively, to enhance the learning capacity of the network.
- The Rectified Linear Unit (ReLU) activation function was applied in the hidden layers to introduce nonlinearity and improve training convergence.
- An output layer with 62 neurons was defined, corresponding to the number of character classes, along with the Softmax activation function for multiclass classification.
- Categorical Cross-Entropy was selected as the loss function to measure classification error.
- The Adam optimizer was used for training, as it adaptively adjusts the learning rate for faster and more stable convergence.
- The network was trained using mini-batch gradient descent, and model weights were updated using the backpropagation algorithm.

- During training, loss values per epoch were recorded to analyze the convergence behavior of the model.

MLP Results

The trained MLP model was evaluated using both quantitative metrics and visual performance analysis.

Confusion Matrix

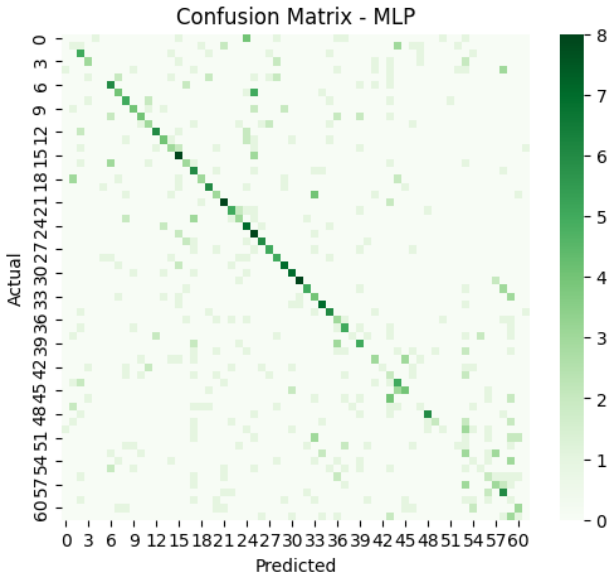

Figure 4: Confusion Matrix for MLP Model

The confusion matrix shows significantly fewer misclassifications compared to the PLA model, indicating improved class separation.

ROC Curves (Micro and Macro Average)

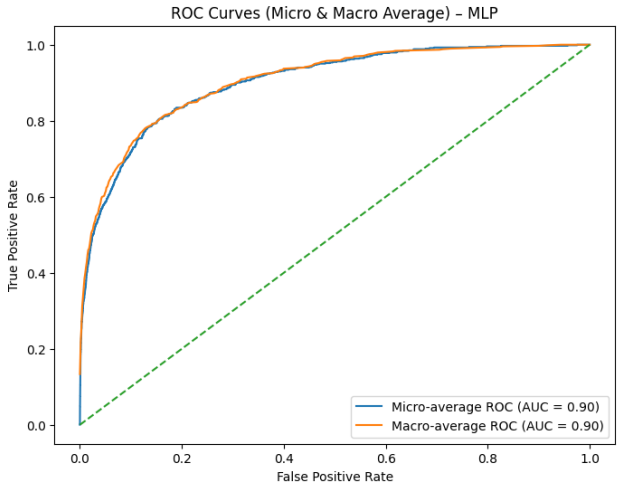

Figure 5: ROC Curve (Micro and Macro Average) for MLP

MLP Training Convergence Curve

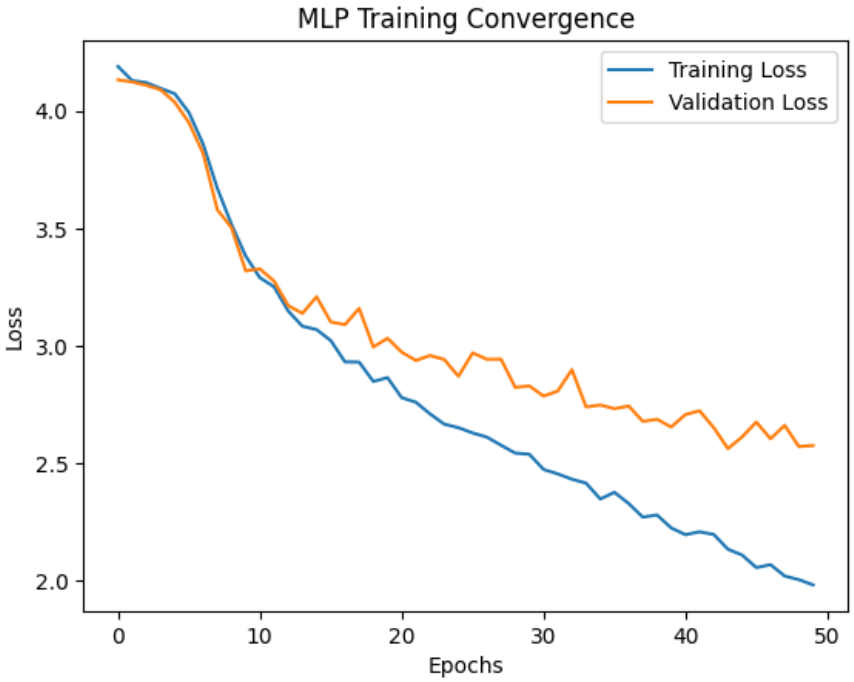

Figure 6: Training and Validation Loss vs Epochs for MLP

The training and validation loss curves exhibit smooth convergence across epochs, demonstrating effective learning and good generalization capability of the MLP model.

6. Performance Tables

Table 1: Overall Performance Comparison between PLA and MLP

Model	Accuracy (%)	Precision	Recall	F1-score
PLA (OvR)	15.39	0.2187	0.1539	0.1175
MLP (Tuned)	33.43	0.3457	0.3343	0.3155

Table 2: Hyperparameter Tuning Results for MLP

Hidden Layers	Activation	Optimizer	Learning Rate	Batch Size	Accuracy (%)
1 (128)	ReLU	SGD	0.01	32	6.60
2 (256–128)	ReLU	Adam	0.001	64	1.61
3 (512–256–128)	Tanh	Adam	0.0005	64	27.13

Table 3: Training Convergence Comparison

Model	Epochs	Final Training Loss	Convergence Behavior
PLA	50	0.0203	Non-convergent
MLP (Tuned)	50	2.1960	Stable convergence

7. A/B Experiment Comparison

Final Tuned Hyperparameters for MLP

- **Hidden Layers:** 1 (128 neurons)
- **Activation Function:** ReLU
- **Optimizer:** SGD
- **Learning Rate:** 0.01
- **Batch Size:** 32

Strengths and Weaknesses of PLA vs MLP

The Single-Layer Perceptron (PLA) is simple, computationally efficient, and easy to implement, making it suitable for basic linearly separable classification problems. However, it can only learn linear decision boundaries and performs poorly on complex, nonlinear datasets such as handwritten character images.

The Multilayer Perceptron (MLP), on the other hand, can model nonlinear relationships through hidden layers and nonlinear activation functions. This enables it to achieve better performance on multiclass handwritten character recognition tasks. However, MLP requires careful hyperparameter tuning and higher computational resources.

Effect of Hyperparameter Tuning on Convergence and Accuracy

Hyperparameter tuning had a significant impact on the convergence and accuracy of the MLP model. The selection of an appropriate learning rate ensured stable weight updates, while the choice of optimizer influenced convergence behavior. In this experiment, SGD with a learning rate of 0.01 provided better performance than Adam. Increasing the number of hidden layers did not improve accuracy and sometimes reduced performance due to overfitting and unstable convergence.

8. Observation Questions

Why does PLA underperform compared to MLP?

PLA underperforms because it can only learn linear decision boundaries and cannot capture complex nonlinear patterns. Handwritten character recognition involves intricate feature relationships that PLA cannot model effectively.

Which hyperparameter had the most impact on MLP performance?

The learning rate and optimizer had the most significant impact on MLP performance. Proper tuning of these parameters resulted in better convergence and higher accuracy, whereas increasing the number of hidden layers did not yield performance gains.

How does optimizer choice affect convergence?

The optimizer determines how model weights are updated during training. In this experiment, SGD provided better convergence and accuracy compared to Adam, while Adam showed faster but less stable convergence for deeper networks.

Does increasing hidden layers always improve performance?

No, increasing the number of hidden layers does not always improve performance. Deeper networks may lead to overfitting and unstable convergence, especially when the dataset size is limited.

How can overfitting be detected and mitigated?

Overfitting can be detected when training accuracy is high but test accuracy is low. It can be mitigated by reducing model complexity, tuning hyperparameters, applying regularization or dropout techniques, and using early stopping.